EasyFood - Atividade 6

Implementando um serviço de envio de e-mail

Nesta atividade, vamos evoluir a EasyFood adicionando um serviço reutilizável de envio de e-mail.

Você pode utilizar qualquer API ou provedor de e-mail. O importante é manter a responsabilidade de envio isolada em um serviço compartilhado.

```
Restaurant / Auth / outros módulos
              |
              v
      email.service.js
              |
              v
       API de e-mail
```

Parte 1 - PREPARE

Deixe sua EasyFood no mesmo ponto da atividade anterior

Antes de começar, confirme que a aplicação está funcionando e que a autenticação já foi implementada.

```
src/
├── database/
│   └── prisma.js
├── modules/
│   ├── restaurants/
│   └── auth/
├── app.js
└── server.js
```

Execute:

```
node server.js
```

Confirme que o login funciona e que você consegue obter um token JWT.

Parte 2 - ESCOLHA UMA API DE E-MAIL

Escolha um serviço que permita o envio de e-mails por API. Você pode utilizar MailerSend, Resend, Brevo, SendGrid ou outro provedor.

Crie sua conta no serviço escolhido e obtenha as informações exigidas pela API, como:

```
API KEY
DOMÍNIO ou REMETENTE VERIFICADO
ENDEREÇO DE E-MAIL DO REMETENTE
```

Não coloque a chave da API diretamente no código. Ela deve ficar no arquivo .env.

Parte 3 - CONFIGURE O .env

1. Adicione as configurações do serviço

Os nomes podem variar conforme a API escolhida. Uma organização possível é:

```
EMAIL_API_KEY="sua-chave-aqui"
EMAIL_FROM="no-reply@seudominio.com"
EMAIL_FROM_NAME="EasyFood"
```

Se o provedor exigir domínio, URL da API ou outra configuração, adicione também ao .env.

O arquivo .env não deve ser enviado para o repositório público.

Parte 4 - CRIE O MÓDULO SHARED

2. Crie a pasta compartilhada

Dentro de src/modules/, crie:

```
shared/
```

Depois crie:

```
src/modules/shared/email.service.js
```

A estrutura ficará:

```
src/
└── modules/
    ├── restaurants/
    ├── auth/
    └── shared/
        └── email.service.js
```

O serviço fica em shared porque o envio de e-mail pode ser utilizado por vários módulos da EasyFood.

Parte 5 - INSTALE A BIBLIOTECA NECESSÁRIA

Consulte a documentação da API escolhida. Alguns provedores possuem um SDK próprio; outros podem ser chamados diretamente por HTTP.

Exemplo, se você escolher MailerSend:

```
npm install mailersend
```

Se escolher outro provedor, instale o pacote indicado na documentação dele.

Não é obrigatório usar MailerSend. Utilize a biblioteca ou forma de integração indicada pela API que você escolheu.

Parte 6 - CRIE UM SERVIÇO GENÉRICO

3. Implemente email.service.js

Independentemente da API escolhida, nossa aplicação deve enxergar uma função simples e reutilizável:

```
sendEmail({
  to,
  toName,
  subject,
  header,
  body,
  footer
})
```

A função recebe:

```
to       -> e-mail do destinatário
toName   -> nome do destinatário
subject  -> assunto do e-mail
header   -> título dentro do e-mail
body     -> conteúdo principal
footer   -> encerramento
```

4. Crie a montagem do HTML

Dentro de email.service.js, crie uma função para padronizar o conteúdo:

```
function buildHtml({ header, body, footer }) {
  return `
    <div style="font-family: sans-serif; max-width: 600px; margin: 0 auto;">
      <h2>${header}</h2>

      <div>${body}</div>

      <hr />

      <p style="color: #888; font-size: 12px;">
        ${footer}
      </p>
    </div>
  `;
}
```

5. Implemente sendEmail

Agora crie a função que fará o envio. A integração interna muda conforme a API escolhida, mas a assinatura da função deve permanecer assim:

```
async function sendEmail({
  to,
  toName,
  subject,
  header,
  body,
  footer
}) {
  const html = buildHtml({
    header,
    body,
    footer
  });

  // Aqui entra a chamada para a API escolhida.
  // Use a API KEY e o remetente definidos no .env.
  // Envie: destinatário, assunto e HTML.

  console.log(
    `[EMAIL] Enviado para ${to} - "${subject}"`
  );
}

module.exports = {
  sendEmail
};
```

A parte que chama a API será diferente para cada provedor. Siga a documentação do serviço escolhido, mas mantenha o restante da EasyFood dependente apenas de sendEmail().

Parte 7 - EXEMPLO COM MAILERSEND

Se você optar por MailerSend, o email.service.js pode ficar assim:

```
const {
  MailerSend,
  EmailParams,
  Sender,
  Recipient
} = require("mailersend");

const mailerSend = new MailerSend({
  apiKey: process.env.EMAIL_API_KEY
});

const sentFrom = new Sender(
```

```
  process.env.EMAIL_FROM,
  process.env.EMAIL_FROM_NAME || "EasyFood"
);

function buildHtml({ header, body, footer }) {
  return `
    <div style="font-family: sans-serif; max-width: 600px; margin: 0 auto;">
      <h2>${header}</h2>
      <div>${body}</div>
      <hr />
      <p style="color: #888; font-size: 12px;">
        ${footer}
      </p>
    </div>
  `;
}

async function sendEmail({
  to,
  toName,
  subject,
  header,
  body,
  footer
}) {
  const recipients = [
    new Recipient(to, toName || to)
  ];

  const emailParams = new EmailParams()
    .setFrom(sentFrom)
    .setTo(recipients)
    .setReplyTo(sentFrom)
    .setSubject(subject)
    .setHtml(
      buildHtml({ header, body, footer })
    )
    .setText(
      `${header}\n\n${body}\n\n${footer}`
    );

  await mailerSend.email.send(emailParams);

  console.log(
    `[EMAIL] Enviado para ${to} - "${subject}"`
  );
}

module.exports = {
  sendEmail
};
```

Se você escolheu outro provedor, substitua somente a parte responsável por conversar com a API.

Parte 8 - USE O SERVIÇO NO RESTAURANTE

6. Importe o Email Service

Abra:

```
src/modules/restaurants/restaurant.service.js
```

Adicione:

```
const emailService =
  require("../shared/email.service");
```

7. Envie um e-mail após o cadastro

Depois de criar o restaurante no banco, faça uma chamada ao serviço:

```
try {
  await emailService.sendEmail({
    to: "SEU_EMAIL_DE_TESTE",
    toName: restaurant.name,
    subject: `Bem-vindo à EasyFood, ${restaurant.name}!`,
    header: `Olá, ${restaurant.name}!`,
    body: `
      Seu restaurante foi cadastrado com sucesso
      na EasyFood.<br/>
      Categoria: ${restaurant.category}.
    `,
    footer: "Equipe EasyFood"
  });
} catch (error) {
  console.error(
    "Erro ao enviar e-mail:",
    error.message
  );
}
```

O cadastro do restaurante deve acontecer antes do envio do e-mail. O e-mail é uma ação secundária.

Parte 9 - MANTENHA O CADASTRO INDEPENDENTE

A organização do createRestaurant deve seguir esta ideia:

```
async function createRestaurant(data) {

  // 1. Operação principal
  const restaurant =
    await prisma.restaurant.create({
      data: {
        name: data.name,
        category: data.category,
        rating: data.rating || 0
      }
    });

  // 2. Ação secundária
  try {
    await emailService.sendEmail({
      to: "SEU_EMAIL_DE_TESTE",
      toName: restaurant.name,
      subject: `Bem-vindo à EasyFood, ${restaurant.name}!`,
      header: `Olá, ${restaurant.name}!`,
      body: `
        Seu restaurante foi cadastrado
        com sucesso na EasyFood.
      `,
      footer: "Equipe EasyFood"
    });
  } catch (error) {
    console.error(
      "Erro ao enviar e-mail:",
      error.message
    );
  }

  // 3. Retorno da operação principal
  return restaurant;
}
```

O try/catch impede que uma falha no serviço externo de e-mail faça o cadastro do restaurante falhar.

Parte 10 - TESTE O ENVIO

8. Inicie a aplicação

```
node server.js
```

Como POST /restaurants está protegido, faça login e copie seu JWT.

9. Cadastre um restaurante

Faça:

```
POST http://localhost:3000/restaurants
```

Headers:

```
Content-Type: application/json
Authorization: Bearer SEU_TOKEN
```

Body:

```
{
  "name": "Cantina da Nona",
  "category": "Italiana",
  "rating": 4.7
}
```

Resultado esperado:

```
201 Created
```

Além do restaurante cadastrado, o e-mail deverá chegar ao endereço utilizado no teste.

Parte 11 - USE A MESMA FUNÇÃO NOVAMENTE

Agora envie um segundo e-mail com outro conteúdo, utilizando a mesma sendEmail().

```
try {
  await emailService.sendEmail({
    to: "SEU_EMAIL_DE_TESTE",
    toName: "Comercial EasyFood",
    subject:
      `Novo restaurante cadastrado: ${restaurant.name}`,
    header: "Novo restaurante na plataforma",
    body: `
      Nome: ${restaurant.name}<br/>
      Categoria: ${restaurant.category}<br/>
      Avaliação: ${restaurant.rating}
    `,
    footer:
      "Notificação automática - EasyFood"
  });
} catch (error) {
  console.error(
    "Erro ao notificar o comercial:",
    error.message
  );
}
```

São dois e-mails diferentes, mas os dois utilizam a mesma função sendEmail().

Parte 12 - TESTE UMA FALHA

10. Simule um erro no serviço de e-mail

Temporariamente, coloque como primeira linha de sendEmail():

```
throw new Error(
  "Falha simulada no envio de e-mail"
);
```

Cadastre outro restaurante.

O terminal deverá registrar o erro do e-mail, mas a API deverá continuar retornando:

```
201 Created
```

Depois consulte:

```
GET http://localhost:3000/restaurants
```

O restaurante deverá continuar no banco.

Depois do teste, remova o throw usado para simular a falha.

Parte 13 - FLUXO FINAL

```
POST /restaurants
       |
       v
restaurant.controller.js
       |
       v
restaurant.service.js
       |
       +----> prisma.restaurant.create()
       |          operação principal
       |
       +----> emailService.sendEmail()
                  |
                  v
              API de e-mail
             ação secundária
```

Estrutura final

```
src/
├── database/
│   └── prisma.js
├── modules/
│   ├── restaurants/
│   │   ├── restaurant.routes.js
│   │   ├── restaurant.controller.js
│   │   └── restaurant.service.js
│   ├── auth/
│   │   └── ...
│   └── shared/
│       └── email.service.js
├── app.js
└── server.js
```

Atividade concluída quando

☐ Uma API de envio de e-mail foi escolhida e configurada.

☐ As credenciais estão no .env.

☐ A pasta modules/shared foi criada.

☐ email.service.js foi criado.

☐ Existe uma função genérica sendEmail().

☐ O serviço recebe destinatário, assunto, header, body e footer.

☐ O cadastro de restaurante dispara um e-mail.

☐ A mesma função consegue enviar conteúdos diferentes.

☐ Uma falha no e-mail não impede o cadastro do restaurante.

☐ O restaurante continua sendo salvo no PostgreSQL mesmo se o serviço de e-mail falhar.